Dependency Injection

Topics

- Dependency Injection (DI)
- Benefits
- Ways to achieve DI
- Using Unity

What is DI?

- Stands for Dependency Injection
- a software design pattern which enables the development of **loosely coupled** code.

- A coding pattern in which a class receives the instances of objects it needs (called dependencies) from an external source rather than creating them itself.

Example:-

Consider two classes, A and B.

Let's assume that class A uses the objects of class B.

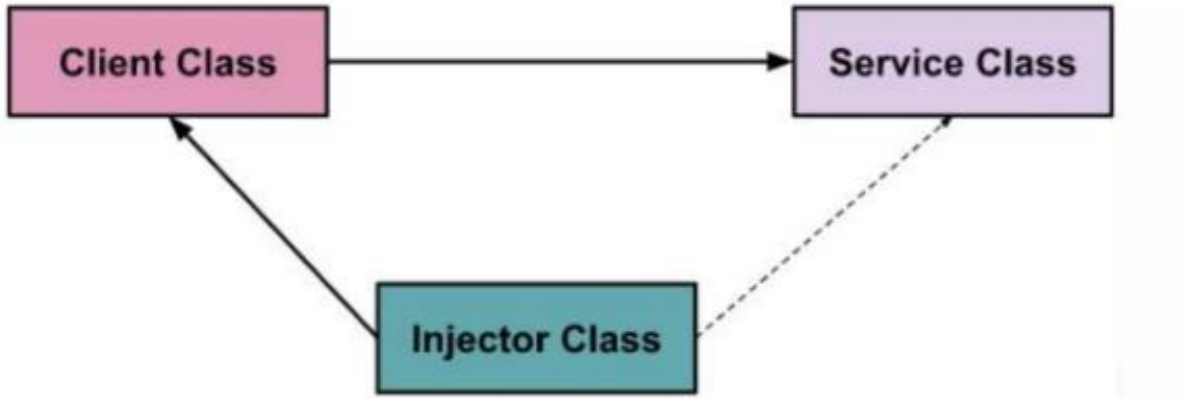
Normally, in OOPS, an instance of class B is created so that class A can access the objects.

Using DI, we move the creation and binding of the dependent objects outside of the class that depend on them.

3 Pillars of DI

Typically, there are three types of classes, they are:

1. Client Class - This is the dependent class, which depends on the service class
2. Service Class - Class that provides the service to the client class.
3. Injector Class - Injects the service class object into the client class.



“DI Achieved using **Interface** type”

Benifits

1. Promotes Loose Coupling

DI reduces direct dependencies between classes, making components easier to change, replace, or update without breaking the system.

2. Improves Code Maintainability

Since dependencies are injected externally, classes are easier to maintain, modify, and extend over time.

3. Enhances Testability

You can easily inject mock or stub implementations of dependencies during unit testing, making tests isolated and reliable.

4. Encourages Interface-Based Design

DI promotes programming to interfaces or abstractions, which is a core principle of object-oriented design.

Benefits

5. Simplifies Object Lifetime Management

DI containers (like Microsoft.Extensions.DependencyInjection) manage the lifecycle (Transient, Scoped, Singleton) of dependencies automatically.

6. Improves Flexibility and Configurability

Dependencies can be swapped easily (e.g., from a SQL database to an in-memory one) by changing the DI configuration.

7. Supports Inversion of Control (IoC)

The flow of control is inverted: objects receive dependencies from the framework rather than creating them internally.

8. Eases Integration with Frameworks

ASP.NET Core is built around DI. Controllers, services, filters, and middleware all support DI out of the box.

Ways to achieve DI

- Constructor(Commonly used)
  - Class receives instance via constructor
- Property
  - Class receives instance via property
- Methods
  - Class receives instance via Method

How Objects are accessed?

Service

==========

```
class Mathclass
{
    public void Add(int x, int y)
    {
        Console.WriteLine(x+y);
    }
}
```

Consumer

==========

```
Mathclass ob = new Mathclass();
Ob.Add(10,10);
```

How Objects are accessed?

```
class A
{
B ob = new B();
   public void hello()
    {
b.welcome();
        Console.WriteLine(“hello called”)
}
}
```

```
class B
{
   public void welcome()
    {
     Console.WriteLine(“welcome called”)
 }
}
```

Drawback

- If you rename class B you need to update in all files
- If you introduce constructor in class B existing code will not work
- Unit Test becomes very complex
- You cannot manage instance
- New Feature update becomes difficult(if C contains better version than B)

How Dependency Injection works?

```
internal interface IMath
{
    void Add(int x, int y);
}

class Mathclass : IMath
{
    public void Add(int x, int y)
    {
        Console.WriteLine(x+y);
    }
}
```

Give me Add Method(I don't care which class)

==========

```
internal class MathClient
{
    IMath m;
    public MathClient(IMath ob)
    {

        m = ob;
    }

    public void show()
    {
        m.Add(10, 10);
    }
}
```

Program.cs

==========

```
IMath o = new Mathclass();

MathClient m = new MathClient(o);
m.show();
```

Property Injection

```
internal interface IMath
{
    void Add(int x, int y);
}

class Mathclass : IMath
{
    public void Add(int x, int y)
    {
        Console.WriteLine(x+y);
    }
}
```

Give me Add Method(I don't care which class)

==========

```
internal class MathClient
{

    public IMath PMath { get; set; }
    public void show()
    {
        PMath.Add(10, 10);
    }
}
```

Program.cs

==========

```
IMath o = new Mathclass();

MathClient m = new MathClient();
m.PMath = o;
m.show();
```

Method Injection

```
internal interface IMath
{
    void Add(int x, int y);
}

class Mathclass : IMath
{
    public void Add(int x, int y)
    {
        Console.WriteLine(x+y);
    }
}
```

Give me Add Method(I don't care which class)

==========

```
internal class MathClient
{

public void show(IMath ob)
    {
        ob.Add(10, 10);
    }
}
```

Program.cs

==========

```
IMath o = new Mathclass();

MathClient m = new MathClient();
m.show(o);
```

Using Unity Package

Install-Package Unity

```
IUnityContainer container = new UnityContainer();
container.RegisterType<IMath, Mathclass>();
//container.RegisterSingleton<IMath,Mathclass>();

// Automatically resolves the dependency for Client
var client = container.Resolve<MathClient>();
client.show();
```